

同济大学计算机科学与技术系  
实验指导



题目：  $\mu$  C/OS-II 系统移植

专    业 计算机科学与技术

课    程 计算机系统试验

授课老师 郭玉臣

时    间 2022 年 2 月

# 目 录

|     |                            |    |
|-----|----------------------------|----|
| 1   | 实验目的.....                  | 3  |
| 2   | 实验内容.....                  | 3  |
| 3   | 实验环境.....                  | 3  |
| 4   | 实验原理.....                  | 3  |
| 5   | 实验过程与方法 .....              | 4  |
| 5.1 | 实现 89 条 MIPS CPU.....      | 4  |
| 5.2 | 增加 WISHBONE 总线.....        | 5  |
| 5.3 | 增加 GPIO 模块 .....           | 10 |
| 5.4 | 增加 UART 控制器.....           | 10 |
| 5.5 | 增加 FLASH 控制器 .....         | 12 |
| 5.6 | 增加 SDRAM 控制器.....          | 15 |
| 5.7 | 利用 UBUNTU 上建立交叉编译环境.....   | 19 |
| 5.8 | 对 MC/OS-II 系统进行改写、编译 ..... | 20 |
| 5.9 | 下板验证 .....                 | 22 |
| 6   | 操作系统编译移植.....              | 27 |
| 7   | 约束文件修改说明.....              | 30 |
| 8   | 实验要求.....                  | 30 |

# 1 实验目的

- 1、回顾编译原理、操作系统等课程；
- 2、贯通计算机相关课程；

# 2 实验内容

按照《自己动手写 CPU》思路，增加 Wishbone 总线，GPIO、UART、Flash 控制器、SDRAM 控制器。利用 Ubuntu 上建立交叉编译环境 对  $\mu\text{C}/\text{OS-II}$  系统进行改写、编译。最终成功将  $\mu\text{C}/\text{OS-II}$  系统移植到 Digilent NEXYS 4 DDR 开发板上并验证。

# 3 实验环境

## 1.1 软件环境

开发环境 Vivado 2020.2

仿真环境 ModelSim PE 10.4c

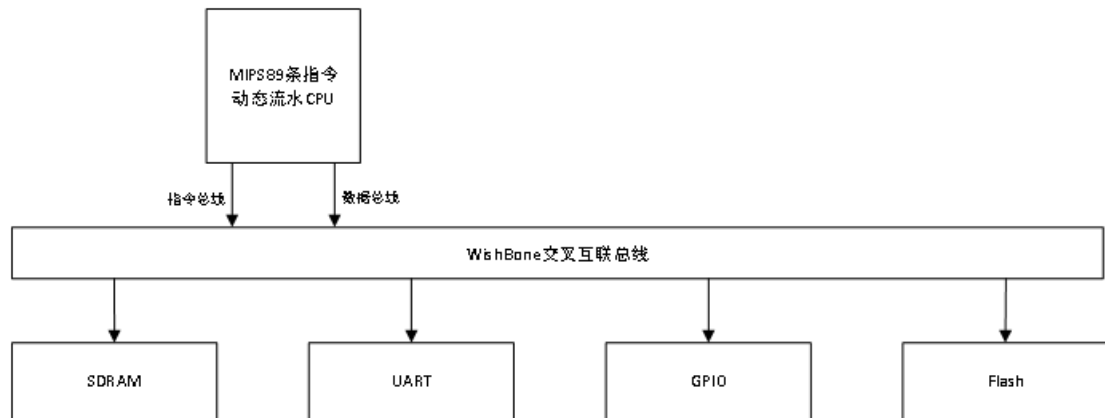
测试环境 MARS 4.5、notepad++

## 1.2 硬件环境

NEXYS 4 DDR Atrix-7

# 4 实验原理

## 1.系统设计总框图



## 5 实验过程与方法

### 5.1 实现 89 条 MIPS CPU

这部分是实验一中的内容。改造 CPU，支持到 89 条指令。参考《自己动手写 CPU》，改造 54 条指令 MIPS 流水线 CPU，支持到 89 条指令，实现 CP0、异常处理，需要新增的 35 条指令如下：

- 移动操作指令：

movn、movz

- 算术操作指令：

clo、madd、maddu、msub、msubu

- 转移指令：

b、bal、bgezal、bgtz、blez、bltz、bltzal

- 加载存储指令：

ll、lwl、lwr、sc、swl、swr

- 异常相关指令：

tge、tgeu、tlb、tlbu、tne、teqi、tgei、tgeiu、tlti、tltiu、tnei

- 其他指令：

nop、ssnop、sync、pref

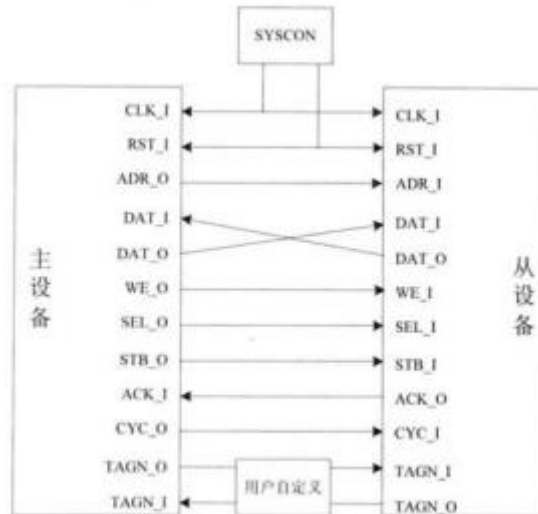
## 5.2 增加 Wishbone 总线

PC 机一般都提供 PCI 插槽，各种板卡（包括显卡、语音卡、网卡，甚至用户自制的板卡）只要满足 PCI 接口标准，就可以直接插在 PC 机的 PCI 插槽使用，十分方便。同样的道理，目前有很多 IP 核的研发者或公司，为了方便不同研发者或公司的 IP 核能够直接连接，就要求这些 IP 核遵守共同的接口标准。在片上系统（SoC）中，处理器核与其他 IP 核通过共享总线互通互联，这些 IP 核必须遵守相同的总线规范。总线规范定义了 IP 核之间的通用接口。目前常见的片上

总线规范有 ARM 公司的 AMBA、IBM 公司的 CoreConnect、Altera 公司的 Avalon，以及这里所说的 Wishbone。

Wishbone 总线规范是 Silicore 公司最先提出的，由于其开放性，现在已有不少用户群，特别是一些免费的 IP 核，大多数都采用 Wishbone 规范。Wishbone 除了开放、免费，还有简单、灵活、轻量、支持用户自定义标签的特点。目前已有 B4 版本的规范，OR1200 中遵循的是 Wishbone B2 与 B3 版本的规范，用户可以自行配置是采用 B2 还是 B3 规范。

Wishbone 有多种互联方式：点对点、数据流、共享总线、交叉互联等。OR1200 内部使用的都是点对点连接方式。在点对点连接方式中，有一个主设备，一个从设备，连接关系如下图所示，图中输出信号使用“\_O”结束，输入信号使用“\_I”结束，同时所有的信号都是高电平有效。



图中主从设备接口的含义如下：

- (1) CLK\_I、RST\_I: 分别是时钟信号、复位信号，由外部输入。
- (2) DAT\_O/DAT\_I: 主设备和从设备的之间的数据信号，数据可以由主设备传送给从设备，也可以由从设备传送给主设备。一对主设备和从设备之间最多存在两条数据总线，一条用于主设备向从设备传输数据，另一条用于从设备向主设备传输数据。
- (3) ADR\_O/ADR\_I: 地址信号，主设备输出地址到从设备。
- (4) WE\_O/WE\_I: 写使能信号，主设备输出到从设备，代表当前周期中进行的操作是写操作还是读操作，1 代表写，0 代表读。
- (5) SEL\_O/SEL\_I: 数据总线选择信号，标识当前操作中数据总线上哪些比特是有效的，以总线粒度为单位。SEL\_O/SEL\_I 的宽度为数据总线宽度除以数据总线粒度。比如一个具有 32 位宽、粒度为 1 个字节的数据总线的选择信号应定义为 SEL\_O(3:0)/SEL\_I(3:0), SEL(4'b1001)代表当前操作中数据总线的最高和最低字节有效。
- (6) CYC\_O/CYC\_I: 总线周期信号，CYC\_O/CYC\_I 有效代表一个主设备请求总线使用权或者正在占有总线，但是不一定正在进行总线操作（是否正在进行总线操作取决于选通信号 STB\_O/STB\_I 是否有效）。只有该信号有效，Wishbone 主设备和从设备之间的其它信号才有意义。CYC\_O/CYC\_I 信

号在一次总线操作过程中必须持续有效，比如一次块读操作可能需要多个时钟周期，那么

CYC\_O/CYC\_I 信号必须在多个时钟周期中持续有效。

(7) STB\_O/STB\_I: 选通信号。选通有效代表主设备发起一次总线操作。只有选通信号有效时(此时 CYC\_O/CYC\_I 也必须为高),ADR\_O/ADR\_I、DAT\_O/DAT\_I、SEL\_O/SEL\_I 才有意义。

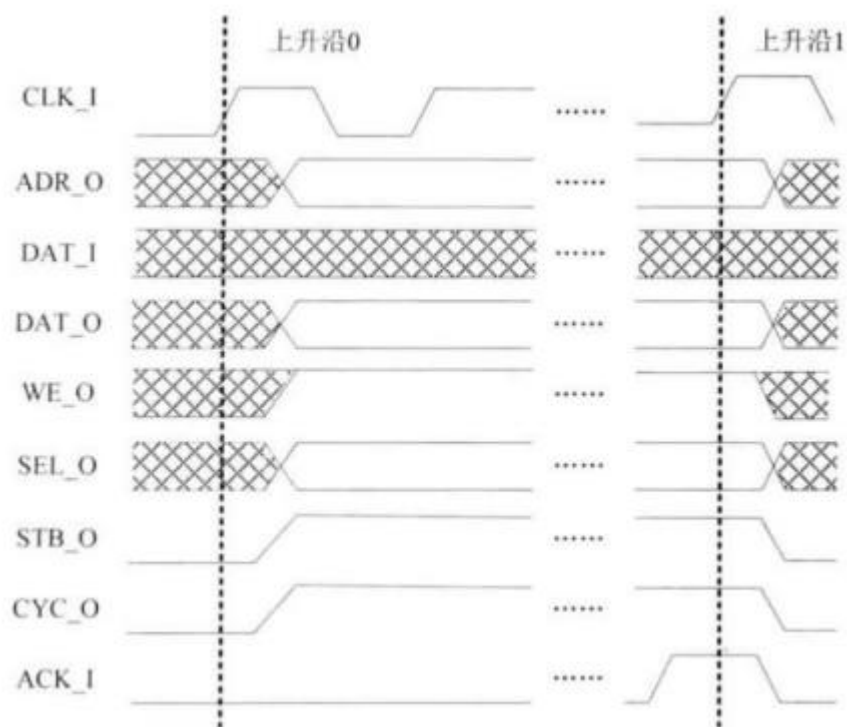
(8) ACK\_O/ACK\_I: 实际还可以有 ERR\_O/ERR\_I、RTY\_O/RTY\_I,都表示主从设备间的操作结束方式信号。ACK 表示成功,ERR 表示错误,RTY 表示重试。操作总是在某一总线周期内完成的,因此操作结束方式也称为总线周期结束方式。成功是操作的正常结束方式,错误表示操作失败,造成失败的原因可能是地址或者数据校验错误,写操作或者读操作不支持等。重试表示从设备当前忙,不能及时处理该操作,可以稍后重新发起。接收到操作失败或者重试后,主设备如何响应取决于主设备的设计者。

(9) TAGN\_O/TAGN\_I: 标签信号,用户可以利用标签信号传递自定义的信息。

一个总线周期由多个不同的时钟周期构成,完成单次读/写操作、块读/写操作、读改写操作,总线周期也相应分为单次读/写周期、块读/写周期、读改写周期。

一般情况下,一次操作由主设备和从设备控制信号的一次握手,以及同时进行的地址和数据总线的一次传输构成。当主设备将 CYC\_O 置高,一个总线周期开始,此后当 STB\_O 为高时,一次总线操作开始。CYC\_O 和 STB\_O 可以同时从低电平变为高电平,表示发起总线周期的同时开始一次总线操作,因此在只有一个主设备时可以将两者合并为一个信号。在 OR1200 中就直接将 CYC\_O、STB\_O 合并成 CYCSTB\_O。

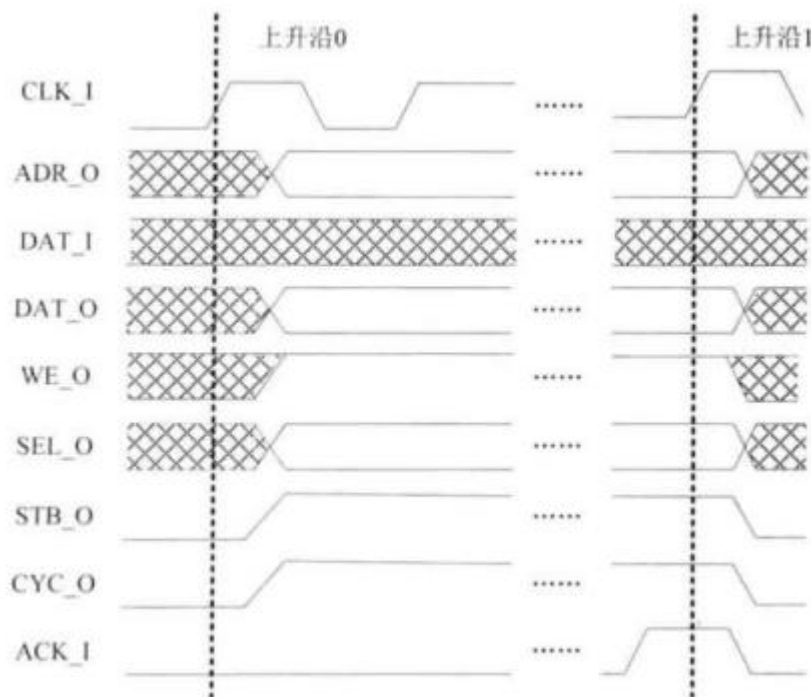
主从设备之间信号虽然很多,但单次读写操作实际上十分简单。单次读操作的 Wishbone 总线信号如下图所示,此处是从主设备的角度观察信号变化。



在时钟上升沿 0，主设备将地址信号 ADR\_O、适当的 SEL\_O 放到总线上，将 WE\_O 置低表示读操作，将 CYC\_O、STB\_O 置高表示一次总线操作开始。在时钟上升沿 1 到达之前，从设备检测到主设备发起的操作，将适当的数据放到主设备的输入信号 DAT\_I，同时将主设备的 ACK\_I 置高作为对主设备 STB\_O 的响应。从设备可以在 ACK\_I 有效之前插入任意数量的等待状态。在时钟上升沿 1，主设备发现 ACK\_I 信号为高，将 DAT\_I 采样，并将 STB\_O 和 CYC\_O 置低表示操作完成。从设备发现 STB\_O 置低后，将主设备的输入信号 ACK\_I 也置低。单次读操作就完成了。

单次写操作的 Wishbone 总线信号如下图所示，此处还是从主设备的角度观察信号变化。





在时钟上升沿 0，主设备将地址信号 ADR\_O、数据信号 DAT\_O 放到总线上，

将 WE\_O 置高表示写操作，将适当的 SEL\_O 放到总线上指示从设备 DAT\_O 中哪些字节是有效的，将 CYC\_O、STB\_O 置高表示一次总线操作开始。

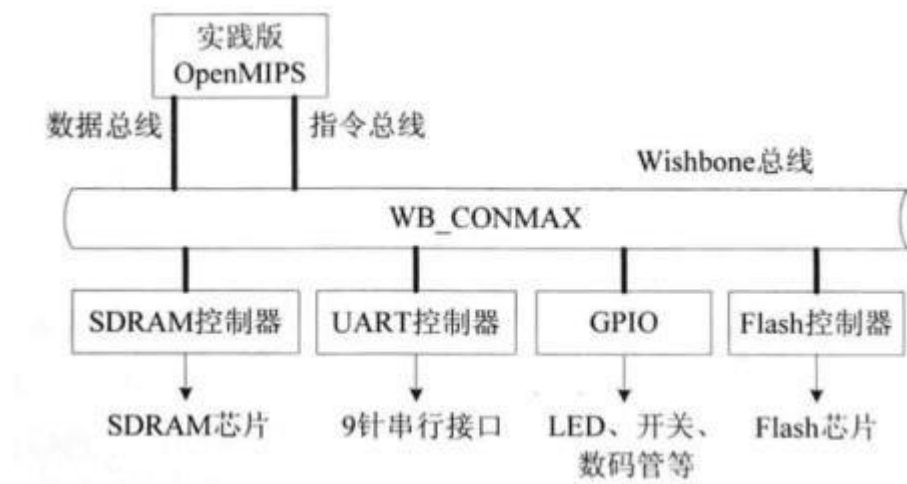
在时钟上升沿 1 到达之前，从设备检测到主设备发起的操作，将锁存 DAT\_O 的数据，同时将主设备的 ACK\_I 置高作为对主设备 STB\_O 的响应。

从设备可以在 ACK\_I 有效之前插入任意数量的等待状态。

在时钟上升沿 1，主设备发现 ACK\_I 信号为高，将 STB\_O 和 CYC\_O 置低表示操作完成。从设备发现 STB\_O 置低后，将主设备的输入信号 ACK\_I 也置低。单次写操作就完成了。

在此次实验中，采用交叉互联方式建立 Wishbone 总线，其结构如下图所示。在 Wishbone 总线上挂接了五个模块：OpenMIPS 处理器、GPIO、UART 控制器、

Flash 控制器、SDRAM 控制器。其中 Wishbone 总线使用的是 OpenCores 站点提供的开源项目 WB\_CONMAX，这是一个 Wishbone 总线互联矩阵，采用的是交叉互联方式，允许多对主从设备同时进行通信。



## 5.3 增加 GPIO 模块

GPIO (General Purpose Input Output) 是以位为单位进行数字输入输出的 IO 接口，作为单纯的通用输入/输出 IO，输入时从外部读取输入信号，输出时将写入的值输出到外部。处理器通过 GPIO 可以与各种设备相连接，例如：LED、开关、七段数码管等。

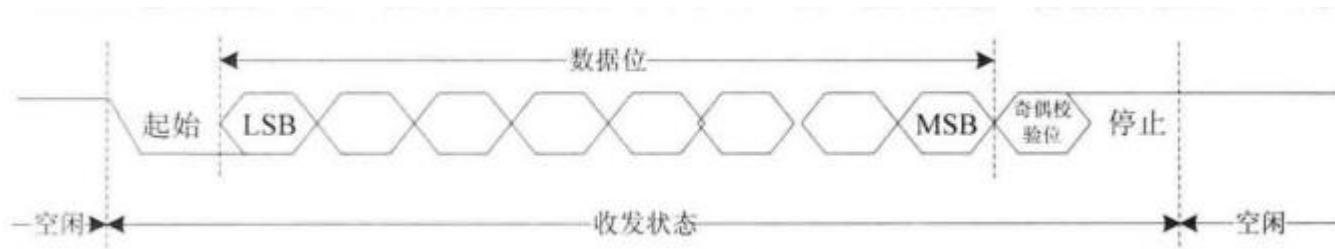
此次实验中，采用月 OpenCores 站点提供的开源项目 GPIO IP Core，添加整个系统中。

## 5.4 增加 UART 控制器

UART 即通用异步收发器 (Universal Asynchronous Receiver/Transmitter)，是广泛使用的串行数据传输协议。它的功能是将并行的数据转变为串行的数据发送或者将接收到的串行数据转变为并行数据。

### 2.4.1 UART 的数据传输

UART 在传输的时候，将待传输数据的每个字符一位一位地传输。传输格式如图下所示。



依次传输起始位、数据位、奇偶校验位、停止位，分别说明如下。

起始位：先发出一个低电平信号，也就是逻辑“0”，表示传输的开始。

数据位：紧接着起始位之后的是数据位。数据位的个数可以是 4、5、6、7、8 等，构成一个字符，从字符的最低位开始传送。

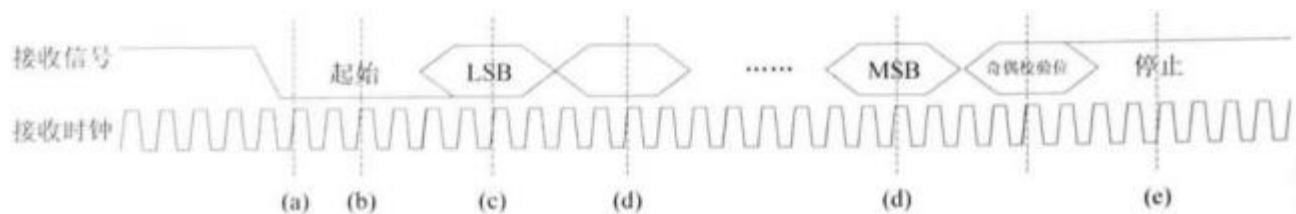
奇偶校验位：数据位之后是奇偶校验位。数据位加上这一位后，使得“1”的个数为偶数（偶校验）或奇数（奇校验），以此来判断数据传送的正确与否。

停止位：是一个字符数据的结束标志。

UART 的通信速率用波特率（baud rate）来表示。波特率指的是信号被调制以后的变化率，即单位时间内载波变化的次数。用于波特率计算的信号除了数据位，还包括起始位、奇偶校验位、停止位，因此，波特率与单纯的数据传输速率是不同的。UART 常用的波特率有 9600 baud、19200 baud、38400 baud 等。

## 2.4.2 UART 的数据接收

UART 的数据接收部分采用比波特率高的采样频率实现。实际使用中，一般使用比波特率高 16 倍的接收时钟进行采样。为了便于说明，下图以接收时钟是波特率的 4 倍为例，给出了数据接收过程。



(a) 当接收信号由高电平变为低电平时，表示检测到起始位。

(b) 检测到起始位后，在接下来的第 2 个时钟周期检查接收信号，如果保持为低电平，说明确实是起始位，开始接收数据。否则认为起始位检测错误，将其忽略。

(c) 确定是起始位后，等待 4 个时钟周期检查接收信号，得到的值就是接收到的第一个 bit，也就是 LSB。

(d) 之后每隔 4 个时钟周期检查接收信号，依次得到传送过来的数据位、奇偶校验位。从图中可以发现，每次采样都是在接收数据的中部，这样采样得到的数据更加准确。

(e) 数据接收完成后，接收停止位。

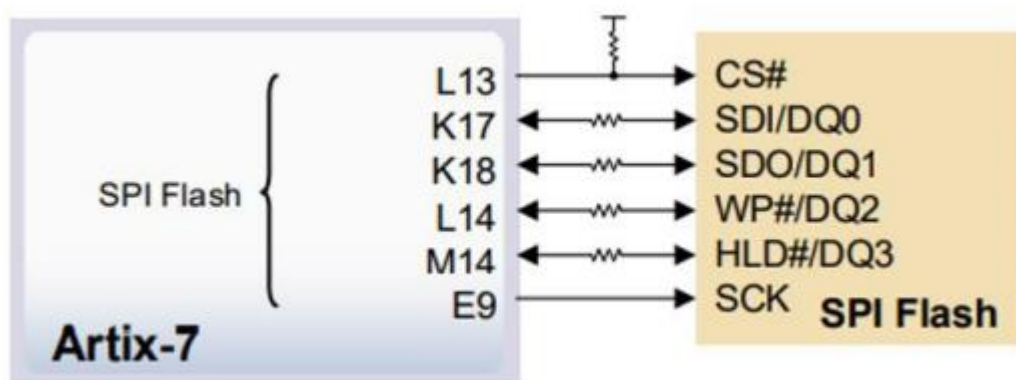
在此次实验中，采用的 UART 控制器是 OpenCores 站点提供的开源项目 UART16550 IP Core，可以在 OpenCores 站点下载源代码。添加了 UART 控制器之后，就可以通过串口与计算机进行通信了。

## 5.5 增加 Flash 控制器

在操作系统移植中，我们需要一个非易失性存储来存放 BootLoader 程序和操作系统的二进制文件，Flash 就充当这样的角色。与《自己动手写 CPU》书上实现的 Flash 不同，此次实验中使用的是 Nexys4 DDR 板载的 SPI FLASH 模块。

在此次实验中，实现了一个一次读取 4Bytes 的 flash 控制器，用来模拟“硬盘”，用于读取只读数据，包括 BootLoader 程序和操作系统的二进制文件。

Nexys4 DDR 使用的 SPI Flash 芯片的具体型号是 S25FL128S 芯片。他的线路图如下：



各接口的具体功能和说明如下：

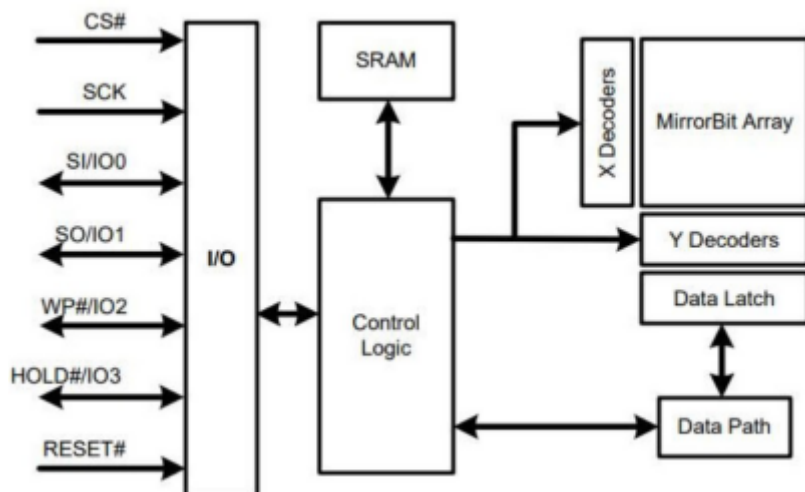
| PAD NO. | PAD NAME    | I/O | FUNCTION                                                |
|---------|-------------|-----|---------------------------------------------------------|
| 1       | /CS         | I   | Chip Select Input                                       |
| 2       | DO (IO1)    | I/O | Data Output (Data Input Output 1)* <sup>1</sup>         |
| 3       | /WP (IO2)   | I/O | Write Protect Input (Data Input Output 2)* <sup>2</sup> |
| 4       | GND         |     | Ground                                                  |
| 5       | DI (IO0)    | I/O | Data Input (Data Input Output 0)* <sup>1</sup>          |
| 6       | CLK         | I   | Serial Clock Input                                      |
| 7       | /HOLD (IO3) | I/O | Hold Input (Data Input Output 3)* <sup>2</sup>          |
| 8       | VCC         |     | Power Supply                                            |

\*1: IO0 and IO1 are used for Standard and Dual SPI instructions

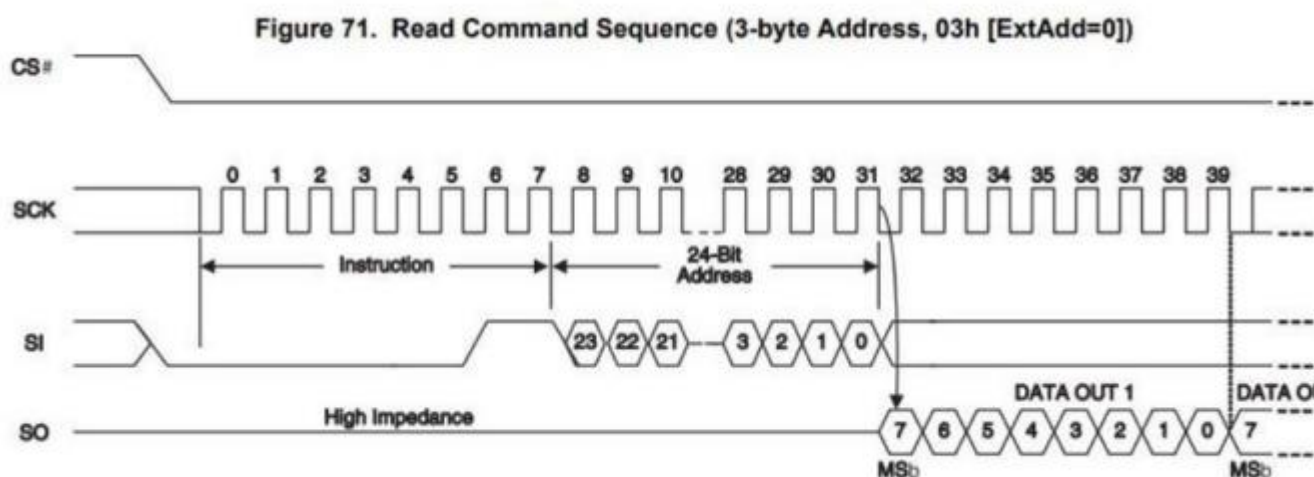
\*2: IO0 – IO3 are used for Quad SPI instructions

spi 协议是一种串行总线，即数据用一根线一位一位的传，这里需要用的有 sck, cs\_n, si, so 四根信号线。sck 是时钟线，用来做信号的同步，频率在工作范围内随意。cs\_n 是片选信号，“\_n”的意思是低电平有效，故当 cs\_n 信号被拉低的时候，这个芯片才开始工作。si 是输入到主设备的信号线，flash 芯片读取数据的时候需要靠它。so 是输入到 flash 的信号线，flash 芯片写入数据的时候需要靠它。

flash 芯片是一种有一堆可读可写的颗粒（EEPROM）和一个控制器组成的，这个控制器封装了许多指令来操作那些颗粒。于是要想使用 flash 芯片，最主要的是对 flash 控制器发出正确的指令。flash 芯片的结构如下图



由于仅仅需要读取数据，此次实验中主要用到了一条指令：READ。根据 S25FL128S 的文档，我们可以看到读取指令的时序为：



从图上看，要想读取数据，需要做的是：

1. 先将 `cs_n` 拉低。
2. 再保持 `sck` 在工作范围内，查阅文档可知，READ 指令最大工作频率是 50MHz，故 `sck` 可选的频率在 (0, 50M] 之间，由于 Nexys4 DDR 的板载晶振频率是 100MHz，于是此次实验中用 50MHz 会比较好分频。
3. 然后通过 `so` 信号线一位一位的输入指令 READ(03H)，读取地址 (24 位)。
4. 最后 flash 芯片就会通过 `si` 信号线一位一位的输出数据。

## 5.6 增加 SDRAM 控制器

SDRAM (Synchronous Dynamic Random Access Memory) 是同步动态随机访问存储器，同步是指 Memory 工作需要同步时钟，内部命令的发送与数据的传输都以它为基准；动态是指存储阵列需要不断地刷新以保证数据不丢失；随机访问是指数据不是线性依次读写，而是可以自由指定地址进行读/写。

与《自己动手写 CPU》书上实现的 SDRAM 不同，此次实验中使用的是 Nexys4

DDR 板载的 DDR 2 模块。此次实验中，使用 DDR 2 模拟一块一次最小读写宽度

8bits，最大读写宽度 32bits 的内存。

NEYXS 4 DDR 里的 DDR2 SDRAM 型号为 MT47H64M16HR-25:H，由于 DDR 协议比较复杂，Xilinx 提供了一个简化控制 DDR 的内存控制器 IP 核，Memory Interface Generator(MIG)，此次实验中就使用这个 IP 核来进行操作。

DDR RAM 时序一共有以下几个信号：

- app\_addr [26:0] in 数据地址
- app\_cmd [2:0] in 操作指令，3'b001 为读，3'b000 为写
- app\_en in 指令使能
- app\_rdy out 指令接受，SDRAM 接受到信号之后就会将其拉高
- app\_wdf\_data [127:0] in 写数据，通过 app\_wdf\_mask 处理后写入 SDRAM
- app\_wdf\_mask [15:0] in 写数据中一个 byte 对应这里一个 bit，将某一位设

高就意味着忽略该位对应的数据 byte 后写入 SDRAM

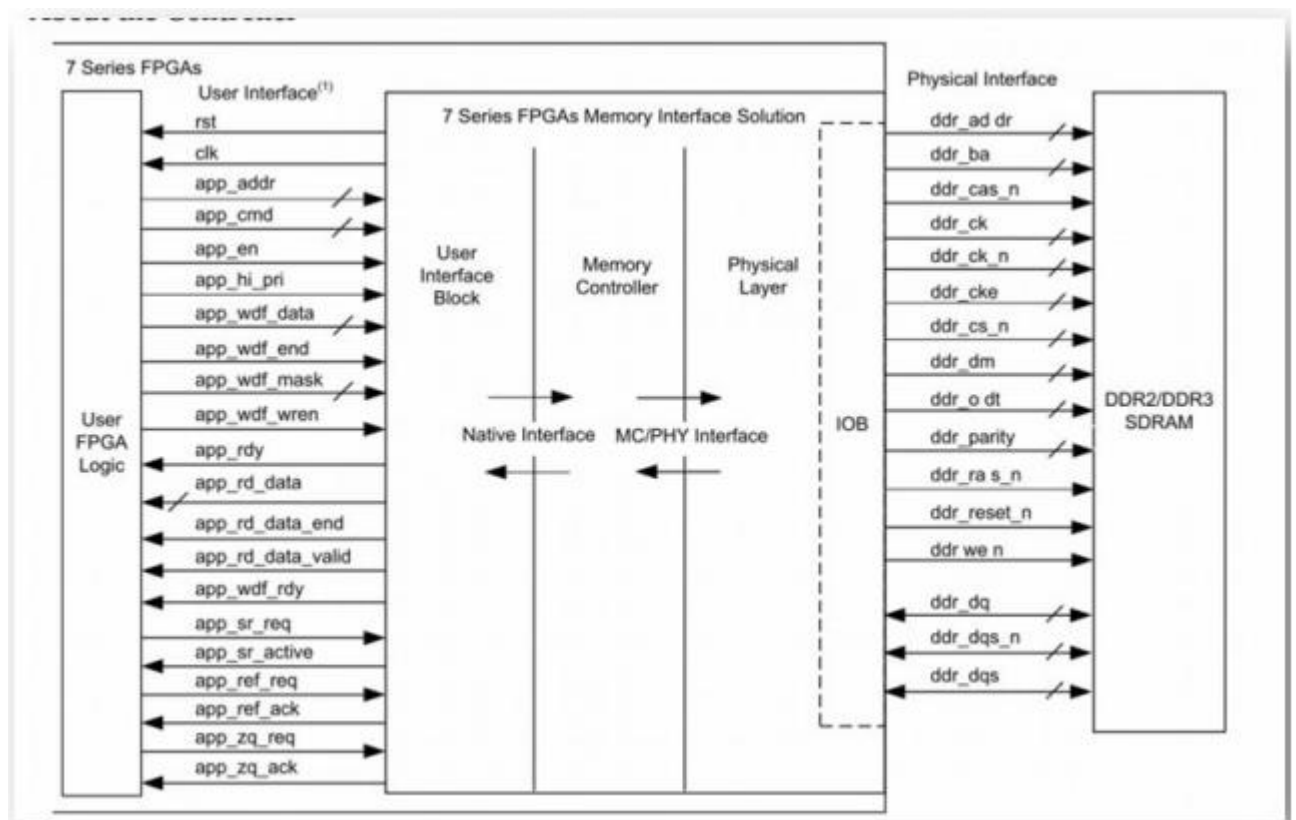
- app\_wdf\_end in 在发送最后一个数据的时候将其拉高，以通知 SDRAM 结束写入操作

- app\_wdf\_wren in 写使能，在把前面的 app\_wdf\_data 和 app\_wdf\_mask 都写好了之后，把这个信号拉高，来通知 SDRAM 可以开始写入

- app\_wdf\_rdy out 写数据指令接收，SDRAM 接收到 app\_wdf\_data 和 app\_wdf\_mask 之后就会将其拉高

- app\_rd\_data [127:0] out 读取的数据

- app\_rd\_data\_valid out 读取数据有效,当 SDRAM 将数据读取到 app\_rd\_data 时会将这个信号拉高
- app\_sr\_req in 保留引脚, 应该拉低
- app\_ref\_req in 拉高这个信号发送一个 refresh 指令给 DRAM, 一次只能拉高一个时钟周期
- app\_zq\_req in 拉高这个信号发送一个 ZQ 校准指令, 校准 ODT 电阻值, 也是一次只能拉高一个时钟周期
- app\_sr\_active out 保留引脚
- app\_ref\_ack out SDRAM 响应 refresh 信号时拉高
- app\_zq\_ack out SDRAM 响应 ZQ 校准信号时拉高
- ui\_clk out IP 核给用户用的时钟, 对于这个 IP 核操作的时序电路必须用这个时钟信号
- ui\_clk\_sync\_rst out 给用户用的复位信号, 同上, 对于这个 IP 核操作的时序电路必须用这个时钟信号



DDR 是 Nexys4 板上的 Nexys4 板子上的一个资源, 容量为 128M。DDR



的操作非常复杂，需要借助一个 IP 核 Memory Interface Generator，简称 MIG（当然用了 MIG 还是非常复杂，需要多级封装）。

MIG 能够封装 DDR 的物理层信号，用户不需要对物理信号有所了解，只需要关注应用信号。MIG 需要按照资料中指导的操作生成。但是由于 DDR 相较于 cpu 属于慢速的设备，所以要协调工作的话还要靠下面的 MIG 控制器。IP 核生成后，需要将 DDR 的物理接口添加到顶层模块，但无需再在 XDC 文件中配置端口，因为生成过程中已经配置好了。

FPGA 如果需要对 DDR 进行读写，则需要一个 DDR 的控制器。DDR 控制器的时序主要有三：

(1) 控制信号：

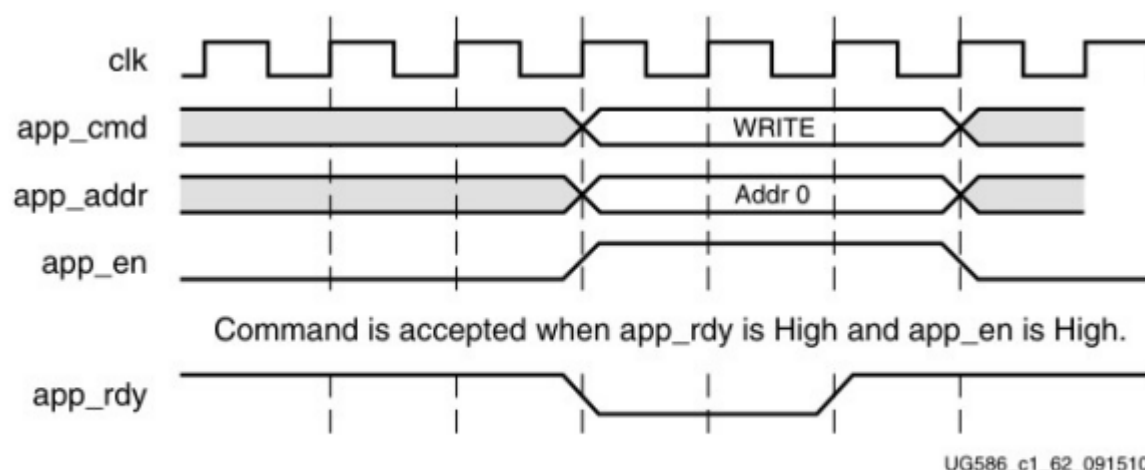


Figure 1-74: UI Command Timing Diagram with app\_rdy Asserted

上图可以看出，只有当 **app\_rdy** 信号有效时，程序所发出的读写命令才会被控制器接收。这点必须注意。

(2) 写操作的时序：

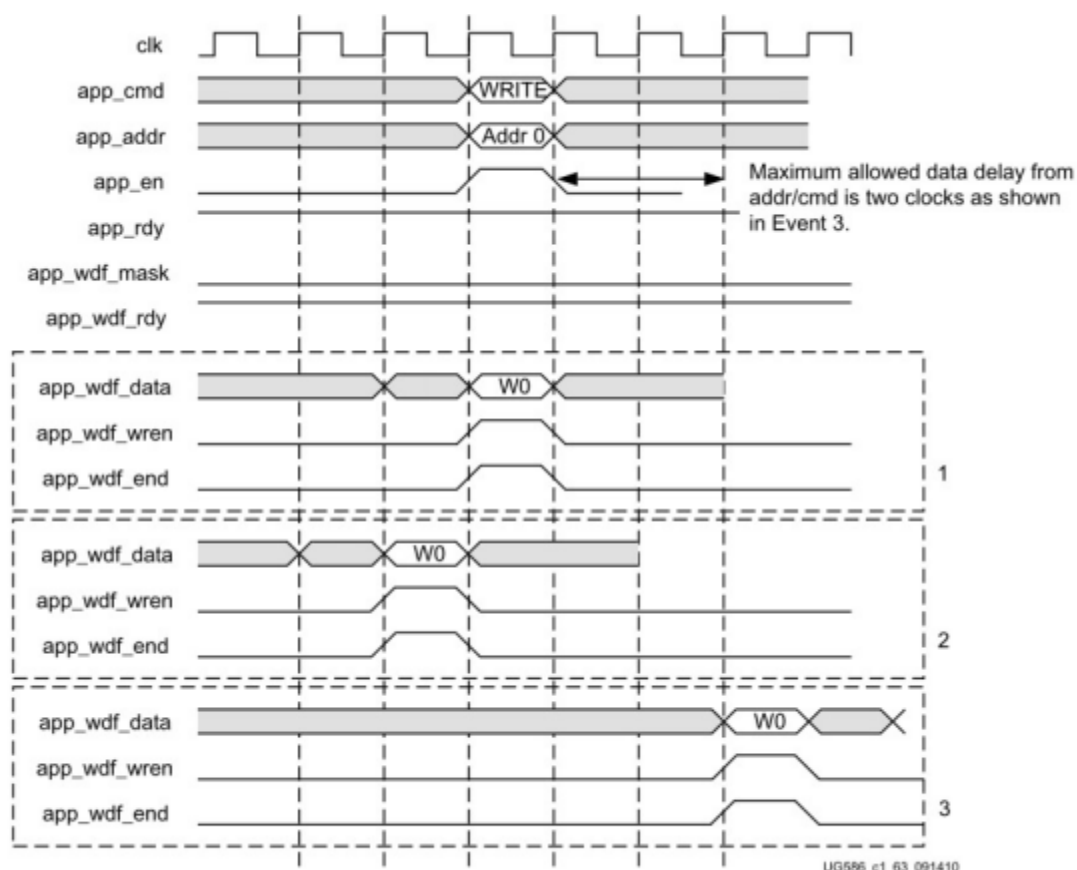


Figure 1-75: 4:1 Mode UI Interface Write Timing Diagram (Memory Burst Type = BL8)

由图可知，在向 DDR 写数据时，需要提供写命令 `app_cmd`、地址 `app_addr`、数据 `app_wdf_data` 等信号，且写入的数据最多可以比 `app_cmd` 提前一个时钟周期有效，最迟可以比 `app_cmd` 晚两个时钟周期有效。在写数据的时候必须检测 `app_rdy` 和 `app_wdf_rdy` 信号是否同时有效，否则写入命令无法成功写入到 DDR 控制器的命令 FIFO 中，从而导致写操作失败。

(3) 读操作时序：

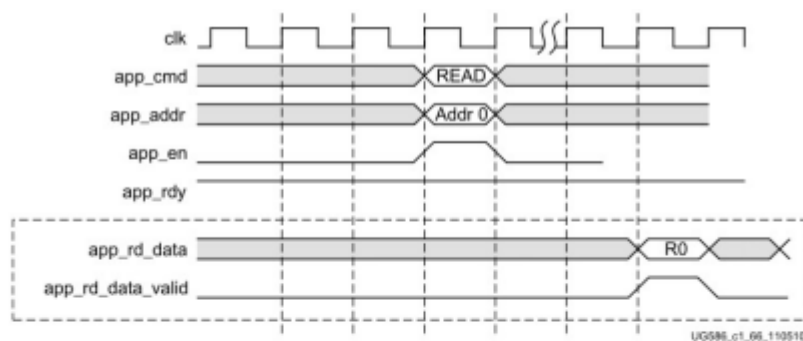


Figure 1-81: 4:1 Mode UI Interface Read Timing Diagram (Memory Burst Type = BL8)

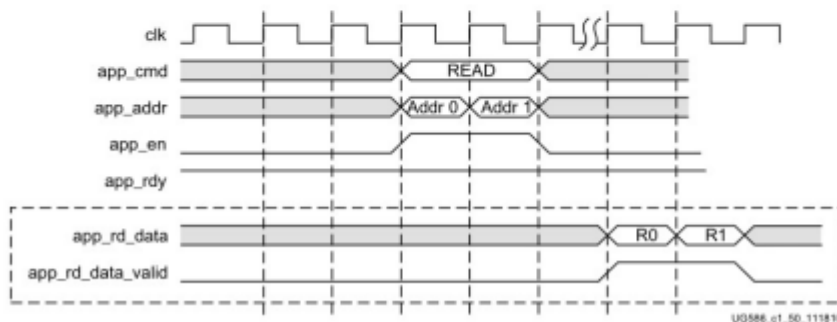


Figure 1-82: 2:1 Mode UI Interface Read Timing Diagram (Memory Burst Type = BL4 or BL8)

读操作的时序比较简单，只需要注意 app\_rdy 是否有效即可，其余不再赘述。

## 5.7 利用 Ubuntu 上建立交叉编译环境

按照《自己动手写 CPU》书上的步骤建立交叉编译环境，使用的环境是 Ubuntu

12.04。按照下述步骤建立 MIPS 编译环境。

在《自己动手写 CPU》附带光盘的 tools 目录下提供了 GNU 工具链安装文件，文件名是 mips-sde-elf-i686-pc-linux-gnu.tar.tar。将安装文件复制到 Ubuntu 的/opt

目录下，打开 Ubuntu 的终端，使用如下命令解压缩：

```
cd /opt
tar vfxj mips-sde-elf-i686-pc-linux-gnu.tar.tar
```

然后打开用户主目录 Home 文件夹，在窗口菜单栏中选择 View->Show Hidden Files，以显示所有文件，这样可以找到一个隐藏文件.bashrc，在此文件的最后加入 PATH 的设置，如下。

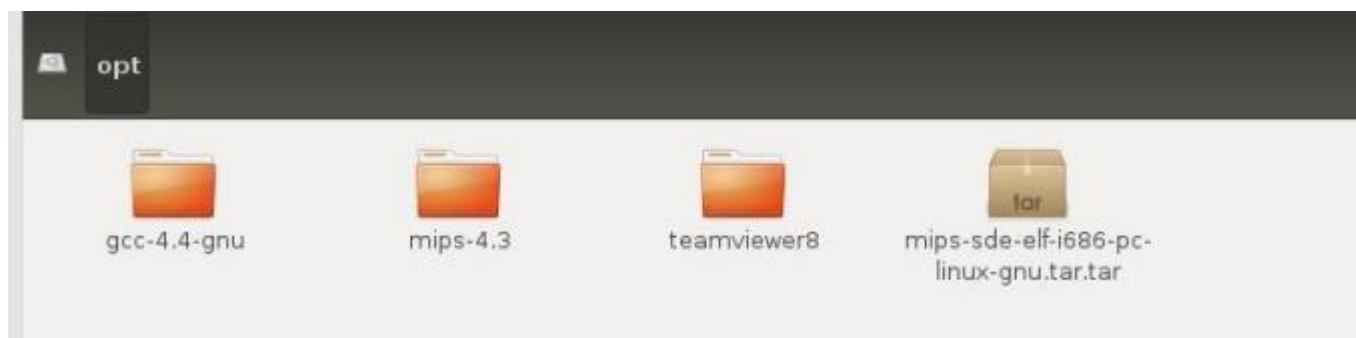
`export PATH="$PATH:/opt/mips-4.3/bin"` 重新

启动 Ubuntu 系统。

重启后，打开终端，在其中输入 `mips-sde-elf-`，然后按两次 `Tab` 键，会列出刚刚安装的，针对 MIPS 平台的所有编译工具，如下图所示，表示 GNU 工具链安装成功。

```
linux@ubuntu: ~  
linux@ubuntu:~$ mips-sde-elf-  
mips-sde-elf-addr2line  mips-sde-elf-gcc          mips-sde-elf-objcopy  
mips-sde-elf-ar         mips-sde-elf-gcc-4.3.2      mips-sde-elf-objdump  
mips-sde-elf-as         mips-sde-elf-gcov          mips-sde-elf-ranlib  
mips-sde-elf-c++        mips-sde-elf-gdb          mips-sde-elf-readelf  
mips-sde-elf-c++filt    mips-sde-elf-gdbtui       mips-sde-elf-run  
mips-sde-elf-conv       mips-sde-elf-gprof        mips-sde-elf-size  
mips-sde-elf-cpp        mips-sde-elf-ld           mips-sde-elf-strings  
mips-sde-elf-g++        mips-sde-elf-nm           mips-sde-elf-strip  
linux@ubuntu:~$ mips-sde-elf-
```

安装成功后，`opt` 目录下文件如下图所示。



## 5.8 对 $\mu\text{C}/\text{OS-II}$ 系统进行改写、编译

首先需要修改 `BootLoader` 程序并重新编译，主要是修改 `UART` 的频率，修改如下：

```

BootLoader.S x
1  .set noat
2  .set noreorder
3  .set nomacro
4
5  .org 0x0
6  .text
7  .align 4
8  .global _start
9  _start:
10
11  lui $1,0x1000
12  ori $1,$1,0x0003
13  ori $2,$0,0x80
14  sb $2,0x0($1)
15
16  lui $1,0x1000
17  ori $1,$1,0x0001
18  ori $2,$0,0x01
19  sb $2,0x0($1)    # MSB of divisor latch
20
21  lui $1,0x1000
22  ori $1,$1,0x0000
23  ori $2,$0,0x45
24  sb $2,0x0($1)    # LSB of divisor latch
25

```

重新编译如下：

```

linux@ubuntu:~/Desktop$ cd bootloader
linux@ubuntu:~/Desktop/bootloader$ make all
mips-sde-elf-as -mips32 BootLoader.S -o BootLoader.o
BootLoader.S: Assembler messages:
BootLoader.S:57: Warning: Macro instruction expanded into multiple instructions
BootLoader.S:58: Warning: Macro instruction expanded into multiple instructions
BootLoader.S:88: Warning: Macro instruction expanded into multiple instructions
BootLoader.S:89: Warning: Macro instruction expanded into multiple instructions
mips-sde-elf-ld -T ram.ld BootLoader.o -o BootLoader.om
mips-sde-elf-objcopy -O binary BootLoader.om BootLoader.bin
mips-sde-elf-objdump -D BootLoader.om > BootLoader.asm
linux@ubuntu:~/Desktop/bootloader$

```

然后进行  $\mu$ C/OS-II 系统的编译，主要用的是《自己动手写 CPU》中提供的系统，在此基础上修改 UART 频率，修改如下：



```

1  #define REG8(addr) *((volatile INT8U *) (addr))
2  #define REG16(addr) *((volatile INT16U *) (addr))
3  #define REG32(addr) *((volatile INT32U *) (addr))
4
5  #define IN_CLK 50000000          /* 输入时钟是27MHz */
6
7  /* ===== */
8
9  /* 串口相关参数、函数 */
10
11  /* ===== */
12
13  #define UART_BAUD_RATE 9600      /* 串口速率是9600bps */
14  #define UART_BASE 0x10000000
15  #define UART_LC_REG 0x00000003 /* Line Control Register */
16  #define UART_IE_REG 0x00000001 /* Interrupt Enable Register */
17  #define UART_TH_REG 0x00000000 /* Transmitter Holding Register */
18  #define UART_LS_REG 0x00000005 /* Line Status Register */
19  #define UART_DLB1_REG 0x00000000 /* Divisor Latch Byte 1 (LSB) */
20  #define UART_DLB2_REG 0x00000001 /* Divisor Latch Byte 2 (MSB) */
21
22  /* Line Status Register的标志位 */
23  #define UART_LS_TEMT 0x40 /* Transmitter empty */
24  #define UART_LS_THRE 0x20 /* Transmit-hold-register empty */
25
26  /* Line Control Register的标志位 */
27  #define UART_LC_NO_PARITY 0x00 /* Parity Disable */
28  #define UART_LC_ONE_STOP 0x00 /* Stop bits: 0x00表示one stop bit */
29  #define UART_LC_WLEN8 0x03 /* Wordlength: 8 bits */
30
31  extern void uart_init(void);
32  extern void uart_putc(char);
33  extern void uart_print_str(char*);
34  extern void uart_print_int(unsigned int);
35

```

编译如下：

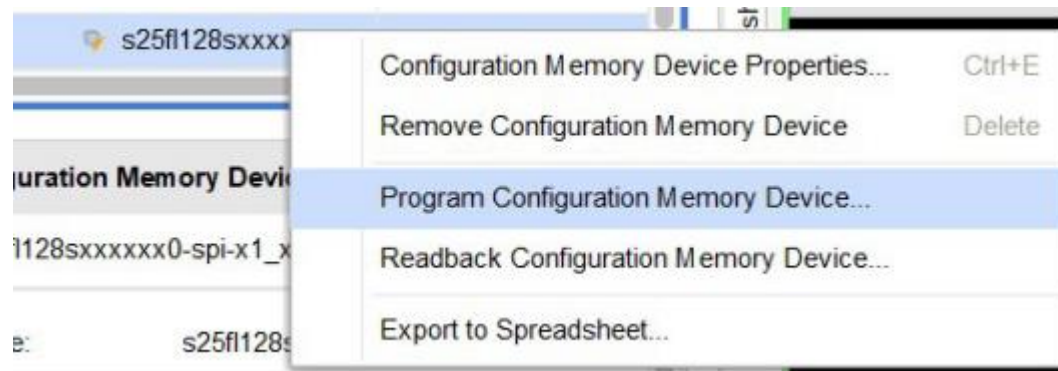
```

linux@ubuntu:~/Desktop/ucosii_OpenMIPS$ make all
make[1]: Entering directory '/home/linux/Desktop/ucosii_OpenMIPS/common'
make[1]: '.depend' is up to date.
make[1]: Leaving directory '/home/linux/Desktop/ucosii_OpenMIPS/common'
make[1]: Entering directory '/home/linux/Desktop/ucosii_OpenMIPS/ucos'
make[1]: '.depend' is up to date.
make[1]: Leaving directory '/home/linux/Desktop/ucosii_OpenMIPS/ucos'
make[1]: Entering directory '/home/linux/Desktop/ucosii_OpenMIPS/port'
make[1]: '.depend' is up to date.
make[1]: Leaving directory '/home/linux/Desktop/ucosii_OpenMIPS/port'
make[1]: Entering directory '/home/linux/Desktop/ucosii_OpenMIPS/common'
make[1]: Nothing to be done for 'all'.
make[1]: Leaving directory '/home/linux/Desktop/ucosii_OpenMIPS/common'
make[1]: Entering directory '/home/linux/Desktop/ucosii_OpenMIPS/ucos'
make[1]: Nothing to be done for 'all'.
make[1]: Leaving directory '/home/linux/Desktop/ucosii_OpenMIPS/ucos'
make[1]: Entering directory '/home/linux/Desktop/ucosii_OpenMIPS/port'
make[1]: Nothing to be done for 'all'.
make[1]: Leaving directory '/home/linux/Desktop/ucosii_OpenMIPS/port'
mips-sde-elf-gcc -Tram.ld -o uc0s00 common/common.o uc0s/uc0s.o port/port.o -nostdlib -lgcc -e 256
mips-sde-elf-objcopy -O binary uc0s00 uc0s00.bin
mips-sde-elf-objdump -D uc0s00.om > uc0s00.asm
./BinMerge.exe -f uc0s00.bin -o OS.bin
linux@ubuntu:~/Desktop/ucosii_OpenMIPS$

```

## 5.9 下板验证

1. 配置 SPI Flash 中的数据为  $\mu$ C/OS-II 操作系统的二进制程序文件



Program Configuration Memory Device ×

Select a configuration file and set programming options.

Memory Device:  ...

Configuration file:  × ...

PRM file:  ...

State of non-config mem I/O pins:  ▼

**Program Operations**

Address Range:  ▼

☒ Erase

☐ Blank Check

☒ Program

☒ Verify

☐ Verify Checksum

**SVF Options**

☐ Create SVF Only (no program operations)

SVF File:  ...

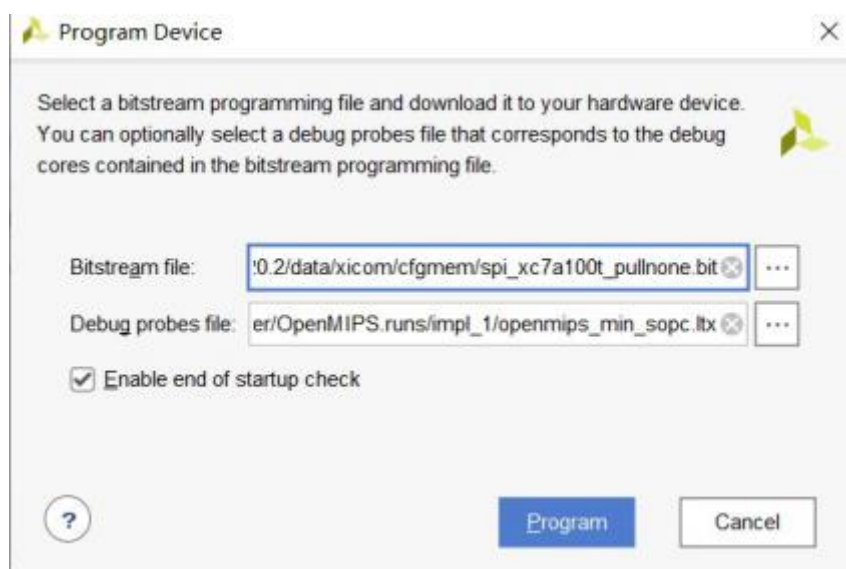
? OK Cancel Apply

2. 打开串口调试程序





### 3. 将生成的 bit 流下板



### 4. 观察开发板现象



## 5. 串口调试程序正确收到数据



## 6 操作系统编译移植

本次实验参照《自己动手写 CPU》进行， $\mu$ C/OS-II 操作系统，boostloader 程序，编译配置文件以及编译工具均使用《自己动手写 CPU》光盘中的资源文件，此步骤只需要对相关文件进行简单修改以适配 NEXYS 4 DDR Atrix-7

### 3.1 boostloader 源码修改

观察到该汇编代码中有许多值为 1 的 label，为了防止错误并且增强程序的可读性，将这些 label 分别改为 L1、L2、L3。

本实验设计的串口波特率为 9600，UART 控制器的时钟频率为 100MHZ，因此分频系数应设置为：28BH。需要 boostloader 汇编程序配置 UART 分频系数寄存器为 28BH。

### 3.2 操作系统源码修改

本实验设计的串口波特率为 9600，UART 控制器的时钟频率为 100MHZ，因此分频系数应设置为：28BH。修改内容位于 openmips.h 文件内需要修改系统时钟频率为 100000000。因为该文件内定义了波特率为 9600 因此应用程序会计算出正确的分频系数。

```
#define IN_CLK 100000000
```

### 3.3 GUN 编译

进入 Ubuntu 的指定文件夹下输入 make all 命令得到以下编译结果：

```
linux@ubuntu:~/Desktop/ucosii_OpenMIPS$ make all
make[1]: Entering directory `/home/linux/Desktop/ucosii_OpenMIPS/common'
make[1]: `depend' is up to date.
make[1]: Leaving directory `/home/linux/Desktop/ucosii_OpenMIPS/common'
make[1]: Entering directory `/home/linux/Desktop/ucosii_OpenMIPS/ucos'
make[1]: `depend' is up to date.
make[1]: Leaving directory `/home/linux/Desktop/ucosii_OpenMIPS/ucos'
make[1]: Entering directory `/home/linux/Desktop/ucosii_OpenMIPS/port'
make[1]: `depend' is up to date.
make[1]: Leaving directory `/home/linux/Desktop/ucosii_OpenMIPS/port'
make[1]: Entering directory `/home/linux/Desktop/ucosii_OpenMIPS/common'
make[1]: Nothing to be done for `all'.
make[1]: Leaving directory `/home/linux/Desktop/ucosii_OpenMIPS/common'
make[1]: Entering directory `/home/linux/Desktop/ucosii_OpenMIPS/ucos'
make[1]: Nothing to be done for `all'.
make[1]: Leaving directory `/home/linux/Desktop/ucosii_OpenMIPS/ucos'
make[1]: Entering directory `/home/linux/Desktop/ucosii_OpenMIPS/port'
```

```

make[1]: Nothing to be done for `all'.
make[1]: Leaving directory `/home/linux/Desktop/ucosii_OpenMIPS/port'
mips-sde-elf-gcc -Tram.ld -o ucosii.om common/common.o ucos/ucos.o port/port.o -nostdlib -lgcc -e 256
mips-sde-elf-objcopy -O binary ucosii.om ucosii.bin
mips-sde-elf-objdump -D ucosii.om > ucosii.asm
./BinMerge.exe -f ucosii.bin -o OS.bin
linux@ubuntu:~/Desktop/ucosii_OpenMIPS$ make all
make[1]: Entering directory `/home/linux/Desktop/ucosii_OpenMIPS/common'
make[1]: `.depend' is up to date.
make[1]: Leaving directory `/home/linux/Desktop/ucosii_OpenMIPS/common'
make[1]: Entering directory `/home/linux/Desktop/ucosii_OpenMIPS/ucos'
make[1]: `.depend' is up to date.
make[1]: Leaving directory `/home/linux/Desktop/ucosii_OpenMIPS/ucos'
make[1]: Entering directory `/home/linux/Desktop/ucosii_OpenMIPS/port'
make[1]: `.depend' is up to date.
make[1]: Leaving directory `/home/linux/Desktop/ucosii_OpenMIPS/port'
make[1]: Entering directory `/home/linux/Desktop/ucosii_OpenMIPS/common'
make[1]: Nothing to be done for `all'.
make[1]: Leaving directory `/home/linux/Desktop/ucosii_OpenMIPS/common'
make[1]: Entering directory `/home/linux/Desktop/ucosii_OpenMIPS/ucos'
make[1]: Nothing to be done for `all'.
make[1]: Leaving directory `/home/linux/Desktop/ucosii_OpenMIPS/ucos'
make[1]: Entering directory `/home/linux/Desktop/ucosii_OpenMIPS/port'
make[1]: Nothing to be done for `all'.
make[1]: Leaving directory `/home/linux/Desktop/ucosii_OpenMIPS/port'
mips-sde-elf-gcc -Tram.ld -o ucosii.om common/common.o ucos/ucos.o port/port.o -nostdlib -lgcc -e 256
mips-sde-elf-objcopy -O binary ucosii.om ucosii.bin
mips-sde-elf-objdump -D ucosii.om > ucosii.asm
./BinMerge.exe -f ucosii.bin -o OS.bin
linux@ubuntu:~/Desktop/ucosii_OpenMIPS$ cd ../
linux@ubuntu:~/Desktop$ cd delaySlot.ucosii_OpenMIPS
bash: cd: delaySlot.ucosii_OpenMIPS: No such file or directory
linux@ubuntu:~/Desktop$ cd delaySlot/ucosii_OpenMIPS
linux@ubuntu:~/Desktop/delaySlot/ucosii_OpenMIPS$ make all
make[1]: Entering directory `/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/common'
mips-sde-elf-gcc -M -I/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/include
-I/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/ucos
-I/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/common -Wall -Wstrict-prototypes
-Werror-implicit-function-declaration -fomit-frame-pointer -fno-strength-reduce -O2 -g -pipe -fno-builtin -nostdlib
-mips32 openmips.c > .depend
make[1]: Leaving directory `/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/common'
make[1]: Entering directory `/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/common'
make[1]: `.depend' is up to date.
make[1]: Leaving directory `/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/common'

```

```

make[1]: Entering directory `/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/ucos'
make[1]: `depend' is up to date.
make[1]: Leaving directory `/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/ucos'
make[1]: Entering directory `/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/port'
mips-sde-elf-gcc -M -I/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/include
-I/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/ucos
-I/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/common -Wall -Wstrict-prototypes
-Werror-implicit-function-declaration -fomit-frame-pointer -fno-strength-reduce -O2 -g -pipe -fno-builtin -nostdlib
-mips32 os_cpu_c.c os_cpu_a.S > .depend
make[1]: Leaving directory `/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/port'
make[1]: Entering directory `/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/port'
make[1]: `depend' is up to date.
make[1]: Leaving directory `/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/port'
make[1]: Entering directory `/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/common'
mips-sde-elf-gcc -I/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/include
-I/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/ucos
-I/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/common -Wall -Wstrict-prototypes
-Werror-implicit-function-declaration -fomit-frame-pointer -fno-strength-reduce -O2 -g -pipe -fno-builtin -nostdlib
-mips32 -c -o openmips.o openmips.c
openmips.c:144: warning: return type of 'main' is not 'int'
mips-sde-elf-ld -r -o common.o openmips.o
make[1]: Leaving directory `/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/common'
make[1]: Entering directory `/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/ucos'
make[1]: Nothing to be done for `all'.
make[1]: Leaving directory `/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/ucos'
make[1]: Entering directory `/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/port'
mips-sde-elf-gcc -I/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/include
-I/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/ucos
-I/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/common -Wall -Wstrict-prototypes
-Werror-implicit-function-declaration -fomit-frame-pointer -fno-strength-reduce -O2 -g -pipe -fno-builtin -nostdlib
-mips32 -c -o os_cpu_a.o os_cpu_a.S
mips-sde-elf-ld -r -o port.o os_cpu_c.o os_cpu_a.o
make[1]: Leaving directory `/home/linux/Desktop/delaySlot/ucosii_OpenMIPS/port'
mips-sde-elf-gcc -Tram.ld -o ucosii.om common/common.o ucos/ucos.o port/port.o -nostdlib -lgcc -e 256
mips-sde-elf-objcopy -O binary ucosii.om ucosii.bin
mips-sde-elf-objdump -D ucosii.om > ucosii.asm
./BinMerge.exe -f ucosii.bin -o OS.bin
linux@ubuntu:~/Desktop/delaySlot/ucosii_OpenMIPS$ vi ucosii.asm

```

```
[1]+  Stopped                  vi ucosii.asm
```

最终得到带 boostloader 的  $\mu$ C/OS-II 操作系统的二进制映像文件：OS.bin

## 7 约束文件修改说明

本实验增加了一些外部管脚，其中与 SDRAM 相关的管脚由 IP 核生成，以下主要罗列 SD 卡的管脚和 UART 的管脚：

| 接口        | 管脚 |
|-----------|----|
| SD_DAT[3] | D2 |
| SD_DAT[2] | F1 |
| SD_DAT[1] | E1 |
| SD_DAT[0] | C2 |
| SD_CD     | A1 |
| SD_CMD    | C1 |
| SD_RESET  | E2 |
| SD_SCK    | B1 |
| TXD       | D4 |
| RXD       | C4 |
| CTS       | D3 |
| RTS       | E5 |

## 8 实验要求

- 1、完成实验，通过所给测试文件测试；
- 2、撰写实验报告